

OPERATING SYSTEMS

Pintos: Report Project #3

Group #6

Davide Frova, Costanza Rodriguez Gavazzi

Fall Semester, March 23, 2024

1 FILES CHANGED

- `/pintos/threads/thread.h` (*modified*)
- `/pintos/threads/thread.c` (*modified*)
- `/pintos/threads/fpr_arith.h` (*added*)

2 CHANGES

`/pintos/threads/thread.h`

- Added import for `threads/fpr_arith.h`
- Added `Nice` constant definitions (min, default, max)
- Added `int nice` and `FPReal recent_cpu` in `struct thread`
- Added method signatures for the new methods added in `/pintos/threads/threads.c`

`/pintos/devices/thread.c`

- (*added*) **Includes:**
 - `threads/fpr_arith.h`
 - `devices/timer.h`
- (*added*) **Global variable:**
 - `FPReal load_avg`, this stores the system load average

- (*modified*): **void thread_tick**; Added the check for the -mlfqs option according to given specs (to handle recent_cpu, priority, load_avg)
- (*modified*): **tid_t thread_create(void)**; Added the check for the -mlfqs option, set default nice value and recent_cpu value. Update priority and check for the priority of the new thread and if necessary, yield current thread.
- (*added*): **void thread_unblock**; We inserted the thread in an ordered way in the **ready_list** instead of just pushing it back. This is to ensure that **ready_list** is ordered such that the head of the list has the highest priority thread.
- (*added*): **static bool compare_priority**; Compares the priorities of two threads t1, t2 and returns true if t1 has a higher priority than t2. We use this when calling **list_insert_ordered**
- (*modified*): **void thread_yield**; We inserted the thread in an ordered way in the **ready_list** instead of just pushing it back. This is to ensure that **ready_list** is ordered such that the head of the list has the highest priority thread.
- (*modified*): **void thread_set_priority**; We update the priority of the current thread. Check if it's necessary to yield comparing current with the top of the **ready_list**. Check for the -mlfqs option (in that case, the method is ignored because in -mlfqs mode the priority is computed based on nice values and recent_cpu);
- (*added*): **void thread_set_nice**; Updates the current thread nice value and updates priority based on that. If necessary, yield.
- (*modified*): **int thread_get_nice**; Get the current thread's nice value.
- (*modified*): **int thread_get_load_avg**; Compute and return the system load average.
- (*modified*): **int thread_get_recent_cpu**; Compute and return the current thread's recent CPU value.
- (*added*): **void update_load_avg**; Computes and sets the system load average.
- (*added*): **thread_action_func * thread_update_recent_cpu**; Computes and updates the recent CPU value of the given thread. Used in the **thread_tick** method, for each thread.
- (*added*): **thread_action_func * update_priority**; Computes and updates the recent thread priority. Check if computed value is in valid range using constants defined previously. Used in the **thread_tick** method, for each thread.